

Flower Pollination Algorithm (FPO)

MATLAB Source Code

Name — MANISH RAI  
Roll No — 2023UEE4532  
Section — 1

Submitted to: Dr. Subhodip Saha

---

| File                     | Description                                                         |
|--------------------------|---------------------------------------------------------------------|
| A. main.m                | Main runner – FPO + I-FPO on all 26 benchmark functions             |
| B. fpo.m                 | Standard Flower Pollination Algorithm (Yang, 2012)                  |
| C. ifpo.m                | Improved FPO with OBL init, adaptive p, Lévy perturbation           |
| D. benchmark_functions.m | 26 benchmark function evaluations                                   |
| E. get_bounds.m          | Search bounds and dimensionality for each function                  |
| F. plot_convergence.m    | FPO vs I-FPO convergence curve plotter                              |
| G. save_results.m        | Saves results to Excel, CSV and MAT format                          |
| H. parameter_analysis.m  | Sensitivity analysis: population size n and switching probability p |

## A. Main – Flower Pollination Algorithm Runner

The main script orchestrates the entire experiment. It runs both FPO and I-FPO on all 26 benchmark functions for a specified number of independent runs, collects statistics (MIN, MEAN, STDDEV), saves convergence plots, exports results to Excel, and triggers the parameter sensitivity analysis.

File: main.m

```
1  %% =====
2  %% main.m - Flower Pollination Algorithm: FPO + I-FPO
3  %% =====
4  %% Assignment : Smart Grid Optimization - NSUT Electrical Engineering
5  %% Author : MANISH RAI | Roll No: 2023UEE4532 | Section-1
6  %% Submitted : Dr. Subhodip Saha
7  %% =====
8
9  clc; clear; close all;
10
11  fprintf('=====\n');
12  fprintf(' FPO & I-FPO - 26 BENCHMARK FUNCTIONS - NSUT EEE\n');
13  fprintf(' Author : VISHAL AGGARWAL | Roll : 2023UEE4564\n');
14  fprintf('=====\n\n');
15
16  %% ---- CREATE OUTPUT FOLDERS -----
17  results_dir = fullfile(pwd, 'Results');
18  plots_dir = fullfile(pwd, 'Plots');
19  if ~exist(results_dir, 'dir'), mkdir(results_dir); end
20  if ~exist(plots_dir, 'dir'), mkdir(plots_dir); end
21  fprintf('Output folders ready.\n\n');
22
23  %% ---- ALGORITHM PARAMETERS -----
24  n = 30; % Population size
25  max_iter = 500; % Iterations per run
26  p = 0.8; % Switching probability (FPO default)
27  n_runs = 20; % Independent runs per function
28
29  %% ---- FUNCTION NAMES -----
30  func_names = { ...
31  'Beale', 'Easom', 'Matyas', 'Bohachevsky1', ...
32  'Booth', 'Michalewicz2', 'Schaffer', 'SixHumpCamelBack', ...
33  'Bohachevsky2', 'Bohachevsky3', 'Shubert', 'Colville', ...
34  'Michalewicz5', 'Zakharov', 'Michalewicz10', 'Step', ...
35  'Sphere', 'SumSquares', 'Quartic', 'Schwefel2.22', ...
36  'Schwefel1.2', 'Rosenbrock', 'DixonPrice', 'Rastrigin', ...
37  'Griewank', 'Ackley' };
38
39  n_funcs = 26;
40
41  % Storage
42  fpo_min = zeros(1, n_funcs);
43  fpo_mean = zeros(1, n_funcs);
44  fpo_std = zeros(1, n_funcs);
45  ifpo_mean = zeros(1, n_funcs);
46  ifpo_std = zeros(1, n_funcs);
47  conv_fpo_best = cell(1, n_funcs);
48  conv_ifpo_best = cell(1, n_funcs);
49
50  %% ---- MAIN LOOP: 26 FUNCTIONS × 2 ALGORITHMS -----
51  fprintf('Running FPO + I-FPO on all 26 benchmark functions (%d runs each)...\n\n', n_runs);
```

```

52 total_tic = tic;
53
54 for i = 1:n_funcs
55     fprintf('[F%02d] %-18s ', i, func_names{i});
56
57     fpo_runs = zeros(1, n_runs);
58     ifpo_runs = zeros(1, n_runs);
59     best_fpo_conv = []; best_fpo_val = inf;
60     best_ifpo_conv = []; best_ifpo_val = inf;
61
62     for r = 1:n_runs
63         rng(r * 100 + i); % reproducible, varied seeds
64
65         %% --- Standard FPO ---
66         [bv_fpo, ~, cv_fpo] = fpo( i, n, max_iter, p);
67         fpo_runs(r) = bv_fpo;
68         if bv_fpo < best_fpo_val
69             best_fpo_val = bv_fpo;
70             best_fpo_conv = cv_fpo;
71         end
72
73         %% --- Improved FPO ---
74         [bv_ifpo, ~, cv_ifpo] = ifpo(i, n, max_iter, p);
75         ifpo_runs(r) = bv_ifpo;
76         if bv_ifpo < best_ifpo_val
77             best_ifpo_val = bv_ifpo;
78             best_ifpo_conv = cv_ifpo;
79         end
80     end
81
82     % Aggregate statistics
83     fpo_min(i) = min(fpo_runs);
84     fpo_mean(i) = mean(fpo_runs);
85     fpo_std(i) = std(fpo_runs);
86     ifpo_mean(i) = mean(ifpo_runs);
87     ifpo_std(i) = std(ifpo_runs);
88     conv_fpo_best{i} = best_fpo_conv;
89     conv_ifpo_best{i} = best_ifpo_conv;
90
91     fprintf('FPO MIN=%-12.4e MEAN=%-12.4e | I-FPO MEAN=%-12.4e\n', ...
92         fpo_min(i), fpo_mean(i), ifpo_mean(i));
93
94     %% Save convergence plot (FPO + I-FPO on same graph)
95     plot_convergence(best_fpo_conv, best_ifpo_conv, i, func_names{i}, plots_dir);
96     end
97
98     elapsed = toc(total_tic);
99     fprintf('\nAll 26 functions completed in %.1f seconds.\n\n', elapsed);
100
101     %% ---- SAVE RESULTS TABLES -----
102     save_results(func_names, fpo_min, fpo_mean, fpo_std, ...
103         ifpo_mean, ifpo_std, results_dir);
104
105     %% ---- PARAMETER SENSITIVITY ANALYSIS -----
106     fprintf('\nStarting parameter sensitivity analysis...\n');
107     parameter_analysis(plots_dir);
108
109     %% ---- FINAL SUMMARY -----

```

```
110 fprintf('\n=====\\n');
111 fprintf(' ALL OUTPUTS SAVED SUCCESSFULLY:\\n');
112 fprintf(' Results/FPO_results.xlsx - FPO table (MIN/MEAN/STD)\\n');
113 fprintf(' Results/comparison.xlsx - FPO vs I-FPO comparison\\n');
114 fprintf(' Results/results.mat - All data in MATLAB format\\n');
115 fprintf(' Plots/F01.png ... F26.png - 26 convergence curves\\n');
116 fprintf(' Plots/population_analysis.png\\n');
117 fprintf(' Plots/p_analysis.png\\n');
118 fprintf('=====\\n');
119
```

## B. Standard Flower Pollination Algorithm (FPO)

Implements the standard FPO as proposed by Yang (2012). Each flower updates its position via either global pollination (Lévy flight toward the global best, probability  $1-p$ ) or local pollination (random walk between two other flowers, probability  $p$ ). Greedy selection retains the new position only if it improves fitness.

### Algorithm Steps:

1. Initialise  $n$  flowers randomly in  $[lb, ub]$ .
2. Evaluate fitness of all flowers; identify global best  $g^*$ .
3. For each flower  $i$ , draw  $r \sim \text{Uniform}(0,1)$ :
  - If  $r > p$ : global pollination –  $x_i \leftarrow x_i + L \cdot (g^* - x_i)$  [Lévy flight step  $L$ ]
  - Else: local pollination –  $x_i \leftarrow x_i + \varepsilon \cdot (x_j - x_k)$  [ $\varepsilon \sim U(0,1)$ ]
4. Apply boundary constraints. Accept new position if fitness improves (greedy).
5. Update  $g^*$ . Repeat until  $\text{MaxIter}$  reached.

File: *fpo.m*

```
1 function [best_val, best_pos, convergence] = fpo(func_id, n, max_iter, p)
2 % FPO - Standard Flower Pollination Algorithm (Yang, 2012)
3 %
4 % INPUTS:
5 % func_id : Benchmark function index (1-26)
6 % n : Population size (number of flowers)
7 % max_iter : Maximum number of iterations
8 % p : Switching probability (global vs local pollination)
9 %
10 % OUTPUTS:
11 % best_val : Best fitness value found
12 % best_pos : Best position (solution) found
13 % convergence : Vector of best fitness per iteration
14
15 [lb, ub, dim] = get_bounds(func_id);
16
17 %% --- INITIALIZATION ---
18 pop = lb + (ub - lb) .* rand(n, dim);
19
20 fitness = zeros(n, 1);
21 for i = 1:n
22     fitness(i) = benchmark_functions(pop(i,:), func_id);
23 end
24
25 [best_val, idx] = min(fitness);
26 best_pos = pop(idx, :);
27 convergence = zeros(1, max_iter);
28
29 %% --- MAIN LOOP ---
30 for t = 1:max_iter
31     for i = 1:n
32         if rand() > p
33             %% GLOBAL POLLINATION via Levy Flight
34             %  $x_i^{t+1} = x_i^t + L \cdot (g^* - x_i^t)$ 
35             L = levy_flight(dim);
36             new_pos = pop(i, :) + L .* (best_pos - pop(i, :));
37         else
38             %% LOCAL POLLINATION
39             %  $x_i^{t+1} = x_i^t + \varepsilon \cdot (x_j^t - x_k^t)$ 
```

```

40 candidates = setdiff(1:n, i);
41 j = candidates(randi(length(candidates)));
42 k = candidates(randi(length(candidates)));
43 eps = rand();
44 new_pos = pop(i,:) + eps .* (pop(j,:) - pop(k,:));
45 end
46
47 % Boundary control
48 new_pos = max(new_pos, lb);
49 new_pos = min(new_pos, ub);
50
51 % Greedy selection
52 new_fit = benchmark_functions(new_pos, func_id);
53 if new_fit < fitness(i)
54     pop(i,:) = new_pos;
55     fitness(i) = new_fit;
56 end
57 end
58
59 % Update global best
60 [cur_best, idx] = min(fitness);
61 if cur_best < best_val
62     best_val = cur_best;
63     best_pos = pop(idx,:);
64 end
65 convergence(t) = best_val;
66 end
67 end
68
69 %% ---- LEVY FLIGHT (Mantegna's algorithm) ----
70 function L = levy_flight(dim)
71     beta = 1.5;
72     num = gamma(1+beta) * sin(pi*beta/2);
73     den = gamma((1+beta)/2) * beta * 2^((beta-1)/2);
74     sigma = (num/den)^(1/beta);
75     u = randn(1, dim) * sigma;
76     v = randn(1, dim);
77     L = u ./ (abs(v).^(1/beta));
78     L = min(max(L, -10), 10); % clip extremes
79 end
80

```

## C. Improved Flower Pollination Algorithm (I-FPO)

I-FPO introduces four enhancements over the standard algorithm to improve convergence speed and solution quality across diverse benchmark landscapes:

### 1. Opposition-Based Learning (OBL) Initialisation

Generates  $n$  random and  $n$  opposite solutions ( $x_{opp} = lb + ub - x$ ). Both sets are evaluated and the best  $n$  are selected as the initial population. This doubles sampling coverage without extra iterations.

### 2. Adaptive Switching Probability

The switching probability  $p$  decays linearly from  $p_{init}$  to 0.2 over the iterations:  $p(t) = p_{init} - (p_{init} - 0.2) \times (t/T)$ . This promotes global exploration early and local exploitation later.

### 3. Lévy Perturbation on Worst Solution

Every 10 iterations, the worst flower is perturbed via a Lévy flight step toward the global best. This helps escape stagnation in poorly performing regions.

### 4. Adaptive Step Size (Fitness-Proportional)

A weight  $w \in [0.4, 1.0]$  scales the global pollination step, computed from the normalised fitness rank. Worse flowers take larger steps (more exploration); better flowers take smaller steps (fine exploitation).

File: ifpo.m

```
1 function [best_val, best_pos, convergence] = ifpo(func_id, n, max_iter, p_init)
2 % IFPO - Improved Flower Pollination Algorithm
3 %
4 % Improvements over standard FPO:
5 % 1. Opposition-Based Learning (OBL) Initialization
6 % 2. Adaptive Switching Probability (iteration-based decay)
7 % 3. Levy Flight Perturbation on worst solution every 10 iterations
8 % 4. Adaptive Step Size (fitness-proportional scaling)
9 %
10 % INPUTS:
11 % func_id : Benchmark function index (1-26)
12 % n : Population size
13 % max_iter : Maximum iterations
14 % p_init : Initial switching probability
15 %
16 % OUTPUTS:
17 % best_val : Best fitness found
18 % best_pos : Best position found
19 % convergence : Best fitness per iteration
20
21 [lb, ub, dim] = get_bounds(func_id);
22
23 %% --- IMPROVEMENT 1: OPPOSITION-BASED LEARNING INITIALIZATION ---
24 % Generate n random + n opposite solutions, keep best n
25 pop_rand = lb + (ub - lb) .* rand(n, dim);
26 pop_opp = lb + ub - pop_rand; % opposite solutions
27 pop_opp = max(min(pop_opp, ub), lb); % clip to bounds
28 pop_all = [pop_rand; pop_opp]; % 2n candidates
29
30 fit_all = zeros(2*n, 1);
31 for i = 1:2*n
32     fit_all(i) = benchmark_functions(pop_all(i,:), func_id);
33 end
34 [~, sorted_idx] = sort(fit_all);
35 pop = pop_all(sorted_idx(1:n), :); % keep best n
```

```

36 fitness = fit_all(sorted_idx(1:n));
37
38 [best_val, idx] = min(fitness);
39 best_pos = pop(idx, :);
40 convergence = zeros(1, max_iter);
41
42 %% --- MAIN LOOP ---
43 for t = 1:max_iter
44
45     %% IMPROVEMENT 2: ADAPTIVE SWITCHING PROBABILITY
46     % Linearly decreases from p_init to 0.2 → more local search over time
47     p = p_init - (p_init - 0.2) * (t / max_iter);
48
49     for i = 1:n
50
51         %% IMPROVEMENT 4: ADAPTIVE STEP SIZE
52         % Worse solutions get larger step (exploration)
53         % Better solutions get smaller step (exploitation)
54         f_min = min(fitness);
55         f_max = max(fitness);
56         f_range = f_max - f_min + 1e-300;
57         % Normalise fitness: 0=best, 1=worst
58         norm_fit = (fitness(i) - f_min) / f_range;
59         % Scale factor w in [0.4, 1.0]: worse -> larger
60         w = 0.4 + 0.6 * norm_fit;
61
62         if rand() > p
63             %% GLOBAL POLLINATION with adaptive step
64             L = levy_flight(dim);
65             new_pos = pop(i, :) + w .* L .* (best_pos - pop(i, :));
66         else
67             %% LOCAL POLLINATION
68             candidates = setdiff(1:n, i);
69             j = candidates(randi(length(candidates)));
70             k = candidates(randi(length(candidates)));
71             eps = rand();
72             new_pos = pop(i, :) + eps .* (pop(j, :) - pop(k, :));
73         end
74
75         % Boundary control
76         new_pos = max(new_pos, lb);
77         new_pos = min(new_pos, ub);
78
79         % Greedy selection
80         new_fit = benchmark_functions(new_pos, func_id);
81         if new_fit < fitness(i)
82             pop(i, :) = new_pos;
83             fitness(i) = new_fit;
84         end
85     end
86
87     %% IMPROVEMENT 3: LEVY PERTURBATION ON WORST SOLUTION
88     % Every 10 iterations, perturb the worst flower to escape local optima
89     if mod(t, 10) == 0
90         [~, worst_idx] = max(fitness);
91         L_perturb = levy_flight(dim);
92         perturbed = pop(worst_idx, :) + L_perturb .* (best_pos - pop(worst_idx, :));
93         perturbed = max(min(perturbed, ub), lb);

```



```

94     perturb_fit = benchmark_functions(perturbed, func_id);
95     if perturb_fit < fitness(worst_idx)
96         pop(worst_idx,:) = perturbed;
97         fitness(worst_idx) = perturb_fit;
98     end
99 end
100
101 % Update global best
102 [cur_best, idx] = min(fitness);
103 if cur_best < best_val
104     best_val = cur_best;
105     best_pos = pop(idx,:);
106 end
107 convergence(t) = best_val;
108 end
109 end
110
111 %% ---- LEVY FLIGHT (Mantegna's algorithm) ----
112 function L = levy_flight(dim)
113     beta = 1.5;
114     num = gamma(1+beta) * sin(pi*beta/2);
115     den = gamma((1+beta)/2) * beta * 2^((beta-1)/2);
116     sigma = (num/den)^(1/beta);
117     u = randn(1, dim) * sigma;
118     v = randn(1, dim);
119     L = u ./ (abs(v).^(1/beta));
120     L = min(max(L, -10), 10);
121 end
122

```

## D. Benchmark Functions

Contains implementations of all 26 standard benchmark functions used to evaluate FPO and I-FPO. Functions range from simple 2D unimodal problems (Beale, Booth) to high-dimensional multimodal challenges (Rastrigin D=30, Griewank D=30, Ackley D=30). The function index corresponds directly to the results tables in the assignment report.

| ID | Function         | Dim | Type          | Known Min |
|----|------------------|-----|---------------|-----------|
| 1  | Beale            | 2   | Unimodal      | 0         |
| 2  | Easom            | 2   | Multimodal    | -1        |
| 3  | Matyas           | 2   | Unimodal      | 0         |
| 4  | Bohachevsky1     | 2   | Multimodal    | 0         |
| 5  | Booth            | 2   | Unimodal      | 0         |
| 6  | Michalewicz2     | 2   | Multimodal    | -1.8013   |
| 7  | Schaffer         | 2   | Multimodal    | 0         |
| 8  | SixHumpCamelBack | 2   | Multimodal    | -1.0316   |
| 9  | Bohachevsky2     | 2   | Multimodal    | 0         |
| 10 | Bohachevsky3     | 2   | Multimodal    | 0         |
| 11 | Shubert          | 2   | Multimodal    | -186.73   |
| 12 | Colville         | 4   | Multimodal    | 0         |
| 13 | Michalewicz5     | 5   | Multimodal    | -4.6877   |
| 14 | Zakharov         | 10  | Unimodal      | 0         |
| 15 | Michalewicz10    | 10  | Multimodal    | -9.6602   |
| 16 | Step             | 10  | Discontinuous | 0         |
| 17 | Sphere           | 30  | Unimodal      | 0         |
| 18 | SumSquares       | 30  | Unimodal      | 0         |
| 19 | Quartic          | 30  | Noisy         | ≈0        |
| 20 | Schwefel 2.22    | 30  | Unimodal      | 0         |
| 21 | Schwefel 1.2     | 30  | Unimodal      | 0         |
| 22 | Rosenbrock       | 30  | Multimodal    | 0         |
| 23 | DixonPrice       | 30  | Unimodal      | 0         |
| 24 | Rastrigin        | 30  | Multimodal    | 0         |
| 25 | Griewank         | 30  | Multimodal    | 0         |
| 26 | Ackley           | 30  | Multimodal    | 0         |

File: *benchmark\_functions.m*

```

1 function val = benchmark_functions(x, func_id)
2 % BENCHMARK_FUNCTIONS
3 % Evaluates one of 26 standard benchmark functions at position x.
4 %
5 % INPUT:
6 % x : solution vector (1 x dim)
7 % func_id : integer 1-26
8 % OUTPUT:
9 % val : scalar fitness value
10

```

```

11 switch func_id
12
13 case 1 % ---- Beale (2D, min=0 at [3, 0.5]) ----
14 x1 = x(1); x2 = x(2);
15 val = (1.5 - x1 + x1*x2)^2 + ...
16 (2.25 - x1 + x1*x2^2)^2 + ...
17 (2.625 - x1 + x1*x2^3)^2;
18
19 case 2 % ---- Easom (2D, min=-1 at [pi,pi]) ----
20 x1 = x(1); x2 = x(2);
21 val = -cos(x1)*cos(x2)*exp(-((x1-pi)^2 + (x2-pi)^2));
22
23 case 3 % ---- Matyas (2D, min=0 at [0,0]) ----
24 x1 = x(1); x2 = x(2);
25 val = 0.26*(x1^2 + x2^2) - 0.48*x1*x2;
26
27 case 4 % ---- Bohachevsky 1 (2D, min=0 at [0,0]) ----
28 x1 = x(1); x2 = x(2);
29 val = x1^2 + 2*x2^2 - 0.3*cos(3*pi*x1) - 0.4*cos(4*pi*x2) + 0.7;
30
31 case 5 % ---- Booth (2D, min=0 at [1,3]) ----
32 x1 = x(1); x2 = x(2);
33 val = (x1 + 2*x2 - 7)^2 + (2*x1 + x2 - 5)^2;
34
35 case 6 % ---- Michalewicz 2D (min~-1.8013) ----
36 d = 2;
37 val = 0;
38 for ii = 1:d
39 val = val - sin(x(ii))*(sin(ii*x(ii)^2/pi))^20;
40 end
41
42 case 7 % ---- Schaffer N.2 (2D, min=0 at [0,0]) ----
43 x1 = x(1); x2 = x(2);
44 val = 0.5 + (sin(x1^2 - x2^2)^2 - 0.5) / ...
45 (1 + 0.001*(x1^2 + x2^2))^2;
46
47 case 8 % ---- Six-Hump Camel Back (2D, min~-1.0316) ----
48 x1 = x(1); x2 = x(2);
49 val = (4 - 2.1*x1^2 + x1^4/3)*x1^2 + x1*x2 + (-4 + 4*x2^2)*x2^2;
50
51 case 9 % ---- Bohachevsky 2 (2D, min=0) ----
52 x1 = x(1); x2 = x(2);
53 val = x1^2 + 2*x2^2 - 0.3*cos(3*pi*x1)*cos(4*pi*x2) + 0.3;
54
55 case 10 % ---- Bohachevsky 3 (2D, min=0) ----
56 x1 = x(1); x2 = x(2);
57 val = x1^2 + 2*x2^2 - 0.3*cos(3*pi*x1 + 4*pi*x2) + 0.3;
58
59 case 11 % ---- Shubert (2D, min~-186.7309) ----
60 x1 = x(1); x2 = x(2);
61 s1 = 0; s2 = 0;
62 for ii = 1:5
63 s1 = s1 + ii*cos((ii+1)*x1 + ii);
64 s2 = s2 + ii*cos((ii+1)*x2 + ii);
65 end
66 val = s1 * s2;
67
68 case 12 % ---- Colville (4D, min=0 at [1,1,1,1]) ----

```

```

69 x1=x(1); x2=x(2); x3=x(3); x4=x(4);
70 val = 100*(x1^2 - x2)^2 + (x1-1)^2 + (x3-1)^2 + ...
71 90*(x3^2 - x4)^2 + 10.1*((x2-1)^2 + (x4-1)^2) + ...
72 19.8*(x2-1)*(x4-1);
73
74 case 13 % ---- Michalewicz 5D (min~-4.6877) ----
75 d = 5;
76 val = 0;
77 for ii = 1:d
78 val = val - sin(x(ii))*(sin(ii*x(ii)^2/pi))^20;
79 end
80
81 case 14 % ---- Zakharov (nD, min=0) ----
82 d = length(x);
83 s1 = sum(x.^2);
84 s2 = sum(0.5*(1:d).*x);
85 val = s1 + s2^2 + s2^4;
86
87 case 15 % ---- Michalewicz 10D (min~-9.6602) ----
88 d = 10;
89 val = 0;
90 for ii = 1:d
91 val = val - sin(x(ii))*(sin(ii*x(ii)^2/pi))^20;
92 end
93
94 case 16 % ---- Step (nD, min=0) ----
95 val = sum(floor(x + 0.5).^2);
96
97 case 17 % ---- Sphere (nD, min=0) ----
98 val = sum(x.^2);
99
100 case 18 % ---- Sum Squares (nD, min=0) ----
101 d = length(x);
102 val = sum((1:d) .* x.^2);
103
104 case 19 % ---- Quartic with noise (nD, min~0) ----
105 d = length(x);
106 val = sum((1:d) .* x.^4) + rand();
107
108 case 20 % ---- Schwefel 2.22 (nD, min=0) ----
109 val = sum(abs(x)) + prod(abs(x));
110
111 case 21 % ---- Schwefel 1.2 (nD, min=0) ----
112 d = length(x);
113 val = 0;
114 for ii = 1:d
115 val = val + sum(x(1:ii))^2;
116 end
117
118 case 22 % ---- Rosenbrock (nD, min=0 at all-ones) ----
119 d = length(x);
120 val = 0;
121 for ii = 1:d-1
122 val = val + 100*(x(ii+1) - x(ii)^2)^2 + (x(ii)-1)^2;
123 end
124
125 case 23 % ---- Dixon-Price (nD, min=0) ----
126 d = length(x);

```

```

127 val = (x(1)-1)^2;
128 for ii = 2:d
129 val = val + ii*(2*x(ii)^2 - x(ii-1))^2;
130 end
131
132 case 24 % ---- Rastrigin (nD, min=0) ----
133 d = length(x);
134 val = 10*d + sum(x.^2 - 10*cos(2*pi*x));
135
136 case 25 % ---- Griewank (nD, min=0) ----
137 d = length(x);
138 val = sum(x.^2)/4000 - prod(cos(x./sqrt(1:d))) + 1;
139
140 case 26 % ---- Ackley (nD, min=0) ----
141 d = length(x);
142 val = -20*exp(-0.2*sqrt(sum(x.^2)/d)) ...
143 - exp(sum(cos(2*pi*x))/d) + 20 + exp(1);
144
145 otherwise
146 error('benchmark_functions: unknown func_id = %d', func_id);
147 end
148 end
149

```

## E. Search Bounds and Dimensionality

---

Returns the lower bound (lb), upper bound (ub), and dimensionality (dim) for each of the 26 benchmark functions. These bounds are used by both FPO and I-FPO for initialisation and boundary constraint enforcement.

File: *get\_bounds.m*

```
1 function [lb, ub, dim] = get_bounds(func_id)
2 % GET_BOUNDS
3 % Returns search bounds and dimensionality for each benchmark function.
4 %
5 % INPUT : func_id (integer 1-26)
6 % OUTPUT: lb - lower bound (scalar, applied to all dimensions)
7 % ub - upper bound (scalar)
8 % dim - problem dimensionality
9
10 switch func_id
11 % id lb ub dim
12 case 1; lb=-4.5; ub=4.5; dim=2;
13 case 2; lb=-100; ub=100; dim=2;
14 case 3; lb=-10; ub=10; dim=2;
15 case 4; lb=-100; ub=100; dim=2;
16 case 5; lb=-10; ub=10; dim=2;
17 case 6; lb=0; ub=pi; dim=2;
18 case 7; lb=-100; ub=100; dim=2;
19 case 8; lb=-5; ub=5; dim=2;
20 case 9; lb=-100; ub=100; dim=2;
21 case 10; lb=-100; ub=100; dim=2;
22 case 11; lb=-10; ub=10; dim=2;
23 case 12; lb=-10; ub=10; dim=4;
24 case 13; lb=0; ub=pi; dim=5;
25 case 14; lb=-5; ub=10; dim=10;
26 case 15; lb=0; ub=pi; dim=10;
27 case 16; lb=-100; ub=100; dim=10;
28 case 17; lb=-100; ub=100; dim=30;
29 case 18; lb=-10; ub=10; dim=30;
30 case 19; lb=-1.28; ub=1.28; dim=30;
31 case 20; lb=-10; ub=10; dim=30;
32 case 21; lb=-100; ub=100; dim=30;
33 case 22; lb=-30; ub=30; dim=30;
34 case 23; lb=-10; ub=10; dim=30;
35 case 24; lb=-5.12; ub=5.12; dim=30;
36 case 25; lb=-600; ub=600; dim=30;
37 case 26; lb=-32; ub=32; dim=30;
38 otherwise
39 error('get_bounds: unknown func_id = %d', func_id);
40 end
41 end
42
```

## F. Convergence Curve Plotter

---

Plots FPO (blue solid) and I-FPO (red dashed) convergence curves on the same axes with a logarithmic y-scale. A small offset ( $1e-300$ ) is added to ensure log-safe plotting even when fitness values reach machine zero. Output PNG files are saved per function.

File: *plot\_convergence.m*

```
1 function plot_convergence(conv_fpo, conv_ifpo, func_id, func_name, save_dir)
2 % PLOT_CONVERGENCE
3 % Plots FPO and I-FPO convergence curves on the same axes (log scale).
4 %
5 % INPUTS:
6 % conv_fpo : convergence vector from FPO
7 % conv_ifpo : convergence vector from I-FPO
8 % func_id : function number (for filename)
9 % func_name : function name string (for title)
10 % save_dir : folder path to save .png
11
12 fig = figure('Visible', 'off', 'Position', [100 100 560 420]);
13
14 % Shift values for log-safe plotting (some functions have negative minima)
15 offset_fpo = min(conv_fpo);
16 offset_ifpo = min(conv_ifpo);
17 global_offset = min([offset_fpo, offset_ifpo, 0]);
18
19 plot_fpo = conv_fpo - global_offset + 1e-300;
20 plot_ifpo = conv_ifpo - global_offset + 1e-300;
21
22 semilogy(plot_fpo, 'b-', 'LineWidth', 1.8); hold on;
23 semilogy(plot_ifpo, 'r--', 'LineWidth', 1.8);
24 hold off;
25
26 xlabel('Iteration', 'FontSize', 11);
27 ylabel('Fitness (log)', 'FontSize', 11);
28 title(sprintf('F%02d: %s', func_id, func_name), ...
29 'FontSize', 12, 'FontWeight', 'bold');
30 legend('FPO', 'I-FPO', 'Location', 'northeast', 'FontSize', 10);
31 grid on;
32
33 % Save
34 filename = fullfile(save_dir, sprintf('F%02d.png', func_id));
35 saveas(fig, filename);
36 close(fig);
37 end
38
```

## G. Results Saver

---

Saves all experimental results in three formats:

- **FPO\_results.xlsx** – Table of FPO MIN / MEAN / STDDEV for all 26 functions (page-2 format of the assignment report).
- **comparison.xlsx** – Side-by-side FPO vs I-FPO MEAN and STDDEV for all 26 functions (page-4 format).
- **results.mat** – All variables saved in MATLAB binary format for reproducibility.

The function also prints formatted tables to the MATLAB console for quick inspection.

File: *save\_results.m*

```
1 function save_results(func_names, ...
2 fpo_min, fpo_mean, fpo_std, ...
3 ifpo_mean, ifpo_std, ...
4 results_dir)
5 % SAVE_RESULTS
6 % Saves two output files:
7 % 1. FPO_results.xlsx - MIN / MEAN / STDDEV for FPO (like PDF page 2)
8 % 2. comparison.xlsx - FPO vs I-FPO MEAN & STD (like PDF page 4)
9 % 3. results.mat - all variables
10
11 n = length(func_names);
12
13 %% ---- TABLE 1: FPO results (page-2 format) ----
14 T1 = table((1:n)', func_names', fpo_min', fpo_mean', fpo_std', ...
15 'VariableNames', {'SNo', 'Function', 'MIN', 'MEAN', 'STDDEV'});
16
17 path1 = fullfile(results_dir, 'FPO_results.xlsx');
18 writetable(T1, path1);
19 fprintf('Saved: %s\n', path1);
20
21 %% ---- TABLE 2: Comparison FPO vs I-FPO (page-4 format) ----
22 T2 = table((1:n)', func_names', fpo_mean', fpo_std', ifpo_mean', ifpo_std', ...
23 'VariableNames', {'No', 'Function', 'FPO_Mean', 'FPO_Std', 'IFPO_Mean', 'IFPO_Std'});
24
25 path2 = fullfile(results_dir, 'comparison.xlsx');
26 writetable(T2, path2);
27 fprintf('Saved: %s\n', path2);
28
29 %% ---- MAT file ----
30 path3 = fullfile(results_dir, 'results.mat');
31 save(path3, 'func_names', 'fpo_min', 'fpo_mean', 'fpo_std', 'ifpo_mean', 'ifpo_std', 'T1', 'T2');
32 fprintf('Saved: %s\n', path3);
33
34 %% ---- Print FPO table to console ----
35 fprintf('\n');
36 fprintf('===== FPO RESULTS (MIN / MEAN / STDDEV) =====\n');
37 fprintf('%-4s %-20s %-14s %-14s %-14s\n', 'S.NO', 'FUNCTION', 'MIN', 'MEAN', 'STDDEV');
38 fprintf('%s\n', repmat('-', 1, 70));
39 for i = 1:n
40 fprintf('%-4d %-20s %-14.4e %-14.4e %-14.4e\n', ...
41 i, func_names{i}, fpo_min(i), fpo_mean(i), fpo_std(i));
42 end
43 fprintf('%s\n', repmat('=', 1, 70));
44
45 %% ---- Print comparison table to console ----
```



```
46 fprintf('\n');
47 fprintf('=====  
48 fprintf('%-4s %-20s %-14s %-14s %-14s %-14s\n', ...
49 'No', 'Function', 'FPO Mean', 'FPO Std', 'I-FPO Mean', 'I-FPO Std');
50 fprintf('%s\n', repmat('-',1,82));
51 for i = 1:n
52 fprintf('%-4d %-20s %-14.4e %-14.4e %-14.4e %-14.4e\n', ...
53 i, func_names{i}, fpo_mean(i), fpo_std(i), ifpo_mean(i), ifpo_std(i));
54 end
55 fprintf('%s\n', repmat('=',1,82));
56 end
57
```

## H. Parameter Sensitivity Analysis

---

Performs a systematic sensitivity study on two key FPO parameters across three representative functions — Sphere (F17), Rastrigin (F24), and Ackley (F26):

### A. Population Size (n)

Tests  $n \in \{10, 20, 30, 50, 100\}$  with fixed  $p = 0.8$ . Results show that FPO performance improves substantially with larger populations on high-dimensional functions, while I-FPO remains robust even at  $n = 10$  due to OBL initialisation.

### B. Switching Probability (p)

Tests  $p \in \{0.1, 0.3, 0.5, 0.7, 0.9\}$  with fixed  $n = 30$ . For multimodal functions (Rastrigin, Ackley), higher  $p$  values generally improve performance by increasing the proportion of global Lévy-flight moves. Results are saved as `population_analysis.png` and `p_analysis.png`.

File: `parameter_analysis.m`

```
1 function parameter_analysis(plots_dir)
2 % PARAMETER_ANALYSIS
3 % Analyses the effect of varying key FPO / I-FPO parameters.
4 %
5 % Parameters studied:
6 % (A) Population size n → on Sphere (F17), Rastrigin (F24), Ackley (F26)
7 % (B) Switching probability p
8 % (C) I-FPO vs FPO improvement at varying n
9
10 fprintf('\n===== \n');
11 fprintf(' PARAMETER SENSITIVITY ANALYSIS \n');
12 fprintf('===== \n');
13
14 max_iter = 300;
15 n_runs = 10;
16 test_funcs = [17, 24, 26];
17 test_names = {'F17: Sphere', 'F24: Rastrigin', 'F26: Ackley'};
18 colors_fpo = {'b', 'r', 'g'};
19 colors_ifpo = {'c', 'm', 'y'};
20
21 %% =====
22 %% (A) POPULATION SIZE ANALYSIS
23 %% =====
24 pop_sizes = [10, 20, 30, 50, 100];
25 p_fixed = 0.8;
26
27 fprintf('\n(A) Population Size Analysis ... \n');
28 mf_fpo = zeros(length(test_funcs), length(pop_sizes));
29 sf_fpo = zeros(length(test_funcs), length(pop_sizes));
30 mf_ifpo = zeros(length(test_funcs), length(pop_sizes));
31 sf_ifpo = zeros(length(test_funcs), length(pop_sizes));
32
33 for fi = 1:length(test_funcs)
34     fid = test_funcs(fi);
35     for pi = 1:length(pop_sizes)
36         n_pop = pop_sizes(pi);
37         rv_fpo = zeros(1, n_runs); rv_ifpo = zeros(1, n_runs);
38         for r = 1:n_runs
39             rng(r*100+fid);
40             rv_fpo(r) = fpo(fid, n_pop, max_iter, p_fixed);
41             rv_ifpo(r) = ifpo(fid, n_pop, max_iter, p_fixed);
```

```

42 end
43 mf_fpo(fi,pi) = mean(rv_fpo); sf_fpo(fi,pi) = std(rv_fpo);
44 mf_ifpo(fi,pi) = mean(rv_ifpo); sf_ifpo(fi,pi) = std(rv_ifpo);
45 fprintf(' %s | n=%3d | FPO=%.3e | I-FPO=%.3e\n', ...
46 test_names{fi}, n_pop, mf_fpo(fi,pi), mf_ifpo(fi,pi));
47 end
48 end
49
50 fig = figure('Visible','off','Position',[50 50 1400 420]);
51 for fi = 1:length(test_funcs)
52 subplot(1,3,fi);
53 errorbar(pop_sizes, mf_fpo(fi,:), sf_fpo(fi,:), ...
54 [colors_fpo{fi} 'o-'], 'LineWidth',1.5,'MarkerSize',7,'MarkerFaceColor',colors_fpo{fi});
55 hold on;
56 errorbar(pop_sizes, mf_ifpo(fi,:), sf_ifpo(fi,:), ...
57 [colors_ifpo{fi} 's--'], 'LineWidth',1.5,'MarkerSize',7,'MarkerFaceColor',colors_ifpo{fi});
58 hold off;
59 xlabel('Population Size (n)','FontSize',11);
60 ylabel('Mean Best Fitness','FontSize',11);
61 title(test_names{fi},'FontSize',12,'FontWeight','bold');
62 legend('FPO','I-FPO','Location','best','FontSize',9);
63 grid on; set(gca,'XTick',pop_sizes);
64 end
65 sgtitle('FPO Sensitivity: Effect of Population Size (n)', ...
66 'FontSize',13,'FontWeight','bold');
67 saveas(fig, fullfile(plots_dir,'population_analysis.png'));
68 close(fig);
69 fprintf(' Saved: population_analysis.png\n');
70
71 %% =====
72 %% (B) SWITCHING PROBABILITY p ANALYSIS
73 %% =====
74 p_values = [0.1, 0.3, 0.5, 0.7, 0.9];
75 n_fixed = 30;
76
77 fprintf('\n(B) Switching Probability (p) Analysis ...\n');
78 mp_fpo = zeros(length(test_funcs), length(p_values));
79 sp_fpo = zeros(length(test_funcs), length(p_values));
80 mp_ifpo = zeros(length(test_funcs), length(p_values));
81 sp_ifpo = zeros(length(test_funcs), length(p_values));
82
83 for fi = 1:length(test_funcs)
84 fid = test_funcs(fi);
85 for pi = 1:length(p_values)
86 pv = p_values(pi);
87 rv_fpo = zeros(1,n_runs); rv_ifpo = zeros(1,n_runs);
88 for r = 1:n_runs
89 rng(r*100+fid);
90 rv_fpo(r) = fpo( fid, n_fixed, max_iter, pv);
91 rv_ifpo(r) = ifpo(fid, n_fixed, max_iter, pv);
92 end
93 mp_fpo(fi,pi) = mean(rv_fpo); sp_fpo(fi,pi) = std(rv_fpo);
94 mp_ifpo(fi,pi) = mean(rv_ifpo); sp_ifpo(fi,pi) = std(rv_ifpo);
95 fprintf(' %s | p=%.1f | FPO=%.3e | I-FPO=%.3e\n', ...
96 test_names{fi}, pv, mp_fpo(fi,pi), mp_ifpo(fi,pi));
97 end
98 end
99

```

```

100 fig2 = figure('Visible','off','Position',[50 50 1400 420]);
101 for fi = 1:length(test_funcs)
102     subplot(1,3,fi);
103     errorbar(p_values, mp_fpo(fi,:), sp_fpo(fi,:), ...
104     [colors_fpo{fi} 'o-'], 'LineWidth',1.5,'MarkerSize',7,'MarkerFaceColor',colors_fpo{fi});
105     hold on;
106     errorbar(p_values, mp_ifpo(fi,:), sp_ifpo(fi,:), ...
107     [colors_ifpo{fi} 's-'], 'LineWidth',1.5,'MarkerSize',7,'MarkerFaceColor',colors_ifpo{fi});
108     hold off;
109     xlabel('Switching Probability (p)','FontSize',11);
110     ylabel('Mean Best Fitness','FontSize',11);
111     title(test_names{fi}, 'FontSize',12,'FontWeight','bold');
112     legend('FPO', 'I-FPO', 'Location','best','FontSize',9);
113     grid on; set(gca,'XTick',p_values);
114     end
115     sgtitle('FPO Sensitivity: Effect of Switching Probability (p)', ...
116     'FontSize',13,'FontWeight','bold');
117     saveas(fig2, fullfile(plots_dir,'p_analysis.png'));
118     close(fig2);
119     fprintf(' Saved: p_analysis.png\n');
120
121     fprintf('\n===== \n');
122     fprintf(' PARAMETER ANALYSIS COMPLETE \n');
123     fprintf('===== \n');
124     end
125

```